

Homework 3: Imbalanced Land-Cover Classification with Logistic Regression

Contents

Scientific Context	1
Learning Objectives	1
Submission	2
0. Setup	2
1. Load the Dataset	2
2. Initial Data Checks	3
3. Quantify Class Imbalance	6
4. Create Train, Validation, and Test Sets	7
5. Build Logistic-Regression Workflows	9
6. Evaluate Models on the Validation Set	12
7. Examine ROC and Precision-Recall Curves	15
8. Test Probability Thresholds Using Validation Data	16
9. Final Evaluation on the Untouched Test Set	21
Grading Rubric (25 points)	22
Reproducibility Checklist	22
Dataset Citation	22

Scientific Context

In this homework, you will work with the **Statlog Landsat Satellite** dataset. Each observation represents a 3 by 3 neighborhood of Landsat multispectral pixels, and the original class label describes the land-cover class of the center pixel.

We will convert the original multiclass problem into an **imbalanced binary-classification** problem:

Can we identify the minority land-cover class, **damp grey soil**, from satellite spectral features?

This is a useful geoscience and environmental-data problem because remote-sensing models are often used to map rare or spatially limited surface conditions. In those settings, ordinary accuracy can be misleading: a model may look accurate while failing to detect the class we care about.

Learning Objectives

By completing this assignment, you will be able to:

1. Load and inspect a public earth-observation dataset in R.
2. Convert a multiclass land-cover problem into an imbalanced binary-classification problem.
3. Quantify class imbalance and explain why accuracy can be misleading.
4. Create stratified train, validation, and test sets.
5. Build leakage-safe preprocessing and logistic-regression workflows with **tidymodels**.
6. Compare a naive baseline, ordinary logistic regression, and upsampled logistic regression.
7. Evaluate imbalanced classifiers using recall, precision, specificity, balanced accuracy, F-scores, ROC-AUC, and PR-AUC.
8. Evaluate whether changing the probability threshold helps, using validation data only.

Submission

Submit this completed R Markdown document knitted to PDF. Replace every **TODO** with your own code or written answer. Keep the test set untouched until the final evaluation section.

0. Setup

This homework uses the modern R machine-learning ecosystem:

- `tidymodels` for splitting, preprocessing, modeling, prediction, and metrics.
- `themis` for imbalance-aware recipe steps such as upsampling.
- `readr` for loading the raw text files.
- `dplyr`, loaded through `tidymodels`, for data handling.

```
# Install packages if needed:
# install.packages(c("tidymodels", "themis", "readr"))

library(tidymodels)
library(themis)
library(readr)
library(ggplot2)

theme_set(theme_minimal())

set.seed(42)
```

1. Load the Dataset

The dataset has 36 numeric predictors. These represent four spectral bands measured across a 3 by 3 pixel neighborhood.

The original land-cover classes are:

Class	Land-Cover Label
1	red soil
2	cotton crop
3	grey soil
4	damp grey soil
5	soil with vegetation stubble
7	very damp grey soil

For this homework, class 4, damp grey soil, is the positive class.

```
feature_names <- paste0("band_", 1:36)
column_names <- c(feature_names, "land_cover")

train_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/satimage/sat.trn"
test_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/satimage/sat.tst"

landsat_raw <- bind_rows(
  read_table(train_url, col_names = column_names, show_col_types = FALSE),
  read_table(test_url, col_names = column_names, show_col_types = FALSE)
)

landsat <- landsat_raw %>%
```

```
mutate(
  damp_grey_soil = if_else(land_cover == 4, "damp_grey_soil", "other"),
  damp_grey_soil = factor(damp_grey_soil, levels = c("damp_grey_soil", "other"))
) %>%
select(-land_cover)
```

```
landsat
```

```
## # A tibble: 6,435 x 37
##   band_1 band_2 band_3 band_4 band_5 band_6 band_7 band_8 band_9 band_10
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     92    115    120     94     84    102    106     79     84    102
## 2     84    102    106     79     84    102    102     83     80    102
## 3     84    102    102     83     80    102    102     79     84     94
## 4     80    102    102     79     84     94    102     79     80     94
## 5     84     94    102     79     80     94     98     76     80    102
## 6     80     94     98     76     80    102    102     79     76    102
## 7     76    102    106     83     76    102    106     87     80     98
## 8     76    102    106     87     80     98    106     79     76     94
## 9     76     89     98     76     76     94     98     76     76     98
## 10    76     94     98     76     76     98    102     72     76     94
## # i 6,425 more rows
## # i 27 more variables: band_11 <dbl>, band_12 <dbl>, band_13 <dbl>,
## #   band_14 <dbl>, band_15 <dbl>, band_16 <dbl>, band_17 <dbl>, band_18 <dbl>,
## #   band_19 <dbl>, band_20 <dbl>, band_21 <dbl>, band_22 <dbl>, band_23 <dbl>,
## #   band_24 <dbl>, band_25 <dbl>, band_26 <dbl>, band_27 <dbl>, band_28 <dbl>,
## #   band_29 <dbl>, band_30 <dbl>, band_31 <dbl>, band_32 <dbl>, band_33 <dbl>,
## #   band_34 <dbl>, band_35 <dbl>, band_36 <dbl>, damp_grey_soil <fct>
```

Question 1: Data Meaning and Prediction Target

Answer in 1-3 sentences.

- What is the model trying to predict?
- Why is this a classification problem rather than a regression problem?

Answer:

TODO:

The model is trying to predict land cover based on the spectral data from the satellite. Each pixel was labeled with a class of land cover and the model is trying to predict which pixels will be damp grey soil based on the observed spectra. Since each band measures the amount of a specific wavelength, and the ratios of these wavelengths determine the color which in turn determines land cover, this can not be a regression issue. Also we are trying to predict a characteristic instead of a measurable value which means it would be a classification model.

2. Initial Data Checks

Before modeling, inspect the dataset. Verify the number of rows and columns, check data types, look for missing values, and summarize the numeric predictors.

```
# TODO: inspect the dataset shape, column names, data types, missing values, and numeric summaries.
dim(landsat) # lots of data, 36 bands??
```

```
## [1] 6435 37
```

```
summary(landsat)
```

```
##      band_1      band_2      band_3      band_4
## Min.   : 39.0   Min.   : 27.00   Min.   : 53.00   Min.   : 33.00
## 1st Qu.: 60.0   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
## Median : 68.0   Median : 87.00   Median :101.00   Median : 81.00
## Mean   : 69.4   Mean    : 83.59   Mean    : 99.29   Mean    : 82.59
## 3rd Qu.: 80.0   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
## Max.   :104.0   Max.    :137.00   Max.    :140.00   Max.    :154.00
##      band_5      band_6      band_7      band_8
## Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.0
## 1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.0
## Median : 68.00   Median : 85.00   Median :101.00   Median : 81.0
## Mean   : 69.15   Mean    : 83.24   Mean    : 99.11   Mean    : 82.5
## 3rd Qu.: 80.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.0
## Max.   :104.00   Max.    :137.00   Max.    :145.00   Max.    :157.0
##      band_9      band_10     band_11     band_12
## Min.   : 40.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
## 1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 68.00
## Median : 67.00   Median : 85.00   Median :100.00   Median : 81.00
## Mean   : 68.91   Mean    : 82.89   Mean    : 98.85   Mean    : 82.39
## 3rd Qu.: 79.00   3rd Qu.:102.00   3rd Qu.:113.00   3rd Qu.: 92.00
## Max.   :104.00   Max.    :130.00   Max.    :145.00   Max.    :157.00
##      band_13     band_14     band_15     band_16
## Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
## 1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
## Median : 68.00   Median : 85.00   Median :101.00   Median : 81.00
## Mean   : 69.29   Mean    : 83.48   Mean    : 99.31   Mean    : 82.64
## 3rd Qu.: 80.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
## Max.   :104.00   Max.    :137.00   Max.    :145.00   Max.    :154.00
##      band_17     band_18     band_19     band_20
## Min.   : 40.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.0
## 1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.0
## Median : 68.00   Median : 85.00   Median :100.00   Median : 81.0
## Mean   : 69.05   Mean    : 83.17   Mean    : 99.15   Mean    : 82.6
## 3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.0
## Max.   :104.00   Max.    :130.00   Max.    :145.00   Max.    :157.0
##      band_21     band_22     band_23     band_24
## Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
## 1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 68.00
## Median : 67.00   Median : 84.00   Median :100.00   Median : 81.00
## Mean   : 68.84   Mean    : 82.86   Mean    : 98.95   Mean    : 82.47
## 3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
## Max.   :104.00   Max.    :130.00   Max.    :145.00   Max.    :157.00
##      band_25     band_26     band_27     band_28
## Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
## 1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
## Median : 68.00   Median : 85.00   Median :100.00   Median : 81.00
## Mean   : 69.16   Mean    : 83.37   Mean    : 99.21   Mean    : 82.66
## 3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
## Max.   :104.00   Max.    :131.00   Max.    :140.00   Max.    :154.00
##      band_29     band_30     band_31     band_32
## Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
## 1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
```

```
## Median : 68.00 Median : 85.00 Median :100.00 Median : 81.00
## Mean : 68.94 Mean : 83.15 Mean : 99.11 Mean : 82.62
## 3rd Qu.: 79.00 3rd Qu.:103.00 3rd Qu.:113.00 3rd Qu.: 92.00
## Max. :104.00 Max. :130.00 Max. :145.00 Max. :157.00
## band_33 band_34 band_35 band_36
## Min. : 39.00 Min. : 27.00 Min. : 50.00 Min. : 29.00
## 1st Qu.: 60.00 1st Qu.: 71.00 1st Qu.: 85.00 1st Qu.: 68.00
## Median : 67.00 Median : 84.00 Median :100.00 Median : 81.00
## Mean : 68.73 Mean : 82.86 Mean : 98.93 Mean : 82.51
## 3rd Qu.: 79.00 3rd Qu.:103.00 3rd Qu.:113.00 3rd Qu.: 92.00
## Max. :104.00 Max. :130.00 Max. :145.00 Max. :157.00
## damp_grey_soil
## damp_grey_soil: 626
## other :5809
##
##
##
##
```

```
any(is.na(landsat)) #false, matches with summary statistics
```

```
## [1] FALSE
```

```
glimpse(landsat) #alls numeric but damp_grey_soil
```

```
## Rows: 6,435
## Columns: 37
## $ band_1 <dbl> 92, 84, 84, 80, 84, 80, 76, 76, 76, 76, 76, 72, 72, 76, ~
## $ band_2 <dbl> 115, 102, 102, 102, 94, 94, 102, 102, 89, 94, 98, 94, 8~
## $ band_3 <dbl> 120, 106, 102, 102, 102, 98, 106, 106, 98, 98, 102, 90, ~
## $ band_4 <dbl> 94, 79, 83, 79, 79, 76, 83, 87, 76, 76, 72, 72, 76, 76, ~
## $ band_5 <dbl> 84, 84, 80, 84, 80, 80, 76, 80, 76, 76, 76, 72, 72, 72, ~
## $ band_6 <dbl> 102, 102, 102, 94, 94, 102, 102, 98, 94, 98, 94, 89, 89~
## $ band_7 <dbl> 106, 102, 102, 102, 98, 102, 106, 106, 98, 102, 90, 94, ~
## $ band_8 <dbl> 79, 83, 79, 79, 76, 79, 87, 79, 76, 72, 76, 76, 76, 72, ~
## $ band_9 <dbl> 84, 80, 84, 80, 80, 76, 80, 76, 76, 76, 76, 72, 76, 68, ~
## $ band_10 <dbl> 102, 102, 94, 94, 102, 102, 98, 94, 98, 94, 89, 89, 94, ~
## $ band_11 <dbl> 102, 102, 102, 98, 102, 102, 106, 102, 102, 90, 94, 98, ~
## $ band_12 <dbl> 83, 79, 79, 76, 79, 79, 79, 76, 72, 76, 76, 76, 76, 72, ~
## $ band_13 <dbl> 101, 92, 84, 84, 84, 76, 80, 80, 80, 76, 76, 76, 76, 71~
## $ band_14 <dbl> 126, 112, 103, 99, 99, 99, 107, 112, 95, 91, 95, 87, 87~
## $ band_15 <dbl> 133, 118, 104, 104, 104, 104, 118, 118, 104, 104, 100, ~
## $ band_16 <dbl> 103, 85, 81, 78, 81, 81, 88, 88, 74, 74, 78, 74, 67, 67~
## $ band_17 <dbl> 92, 84, 84, 84, 76, 76, 80, 80, 76, 76, 76, 76, 71, 68, ~
## $ band_18 <dbl> 112, 103, 99, 99, 99, 99, 112, 107, 91, 95, 91, 87, 87, ~
## $ band_19 <dbl> 118, 104, 104, 104, 104, 108, 118, 113, 104, 100, 100, ~
## $ band_20 <dbl> 85, 81, 78, 81, 81, 85, 88, 85, 74, 78, 74, 67, 70, 67, ~
## $ band_21 <dbl> 84, 84, 84, 76, 76, 76, 80, 80, 76, 76, 76, 71, 71, 68, ~
## $ band_22 <dbl> 103, 99, 99, 99, 99, 103, 107, 95, 95, 91, 87, 87, 83, ~
## $ band_23 <dbl> 104, 104, 104, 104, 108, 118, 113, 100, 100, 100, 100, ~
## $ band_24 <dbl> 81, 78, 81, 81, 85, 88, 85, 78, 78, 74, 74, 70, 67, 67, ~
## $ band_25 <dbl> 102, 88, 84, 84, 84, 84, 79, 79, 75, 75, 79, 75, 75, 71~
## $ band_26 <dbl> 126, 121, 107, 99, 99, 103, 107, 103, 91, 91, 87, 91, 8~
## $ band_27 <dbl> 134, 128, 113, 104, 104, 104, 113, 104, 96, 96, 93, 96, ~
## $ band_28 <dbl> 104, 100, 87, 79, 79, 79, 87, 83, 75, 71, 71, 71, 67, 6~
```

```
## $ band_29      <dbl> 88, 84, 84, 84, 84, 79, 79, 79, 75, 79, 79, 75, 71, 71,~
## $ band_30      <dbl> 121, 107, 99, 99, 103, 107, 103, 103, 91, 87, 87, 87, 8~
## $ band_31      <dbl> 128, 113, 104, 104, 104, 109, 104, 104, 96, 93, 93, 93,~
## $ band_32      <dbl> 100, 87, 79, 79, 79, 87, 83, 79, 71, 71, 67, 67, 67, 62~
## $ band_33      <dbl> 84, 84, 84, 84, 79, 79, 79, 79, 79, 79, 75, 71, 71, 67,~
## $ band_34      <dbl> 107, 99, 99, 103, 107, 107, 103, 95, 87, 87, 87, 87, 79~
## $ band_35      <dbl> 113, 104, 104, 104, 109, 109, 104, 100, 93, 93, 96, 89,~
## $ band_36      <dbl> 87, 79, 79, 79, 87, 87, 79, 79, 71, 67, 71, 67, 62,~
## $ damp_grey_soil <fct> other, other, other, other, other, other, other,~
unique(landsat$damp_grey_soil) # other and damp_grey_soil (this should be yes/no or some other binary c

## [1] other          damp_grey_soil
## Levels: damp_grey_soil other
# Hints: dim, glimpse, colSums, is.na, summary
```

Question 2: Predictor Variables

- Are the predictors mostly numeric or categorical?

Answer:

TODO:

All predictors are numeric, the only categorical variable is the land use being categorized as damp grey soil.

3. Quantify Class Imbalance

Calculate the number and percentage of observations in each class. Then calculate the majority-to-minority ratio.

```
# calculate class counts and percentages.

count(landsat, damp_grey_soil)

## # A tibble: 2 x 2
##   damp_grey_soil      n
##   <fct>          <int>
## 1 damp_grey_soil    626
## 2 other            5809

count(landsat, damp_grey_soil) %>% mutate(percent = n / sum(n) * 100) #9.7 vs 90.3

## # A tibble: 2 x 3
##   damp_grey_soil      n percent
##   <fct>          <int>   <dbl>
## 1 damp_grey_soil    626     9.73
## 2 other            5809    90.3

# calculate the majority-to-minority class ratio.
class_counts <- count(landsat, damp_grey_soil)
max(class_counts$n) / min(class_counts$n)

## [1] 9.279553
```

Question 3: Why Accuracy Can Mislead

Answer in 2-3 sentences.

- If a model predicted the majority class for every row, why might its accuracy look acceptable?

- What would that model's recall be for damp grey soil?
- Why is recall important when mapping a rare class?

Answer:

TODO:

The model predicting the majority class for every row would have a 90% accuracy. That is a really high accuracy and would be considered acceptable because it covers majority of data and could be considered significant if $p=0.1$. The model's recall for the damp grey soil would be 0, it would not have correctly guessed any of the minority class. Recall is important because it measures the ratio between correctly estimated data (true positives) and the true number of cases (true positives and false negatives/missed detections). This shows how great/terrible the model predicted a case that is easy to incorrectly estimate.

4. Create Train, Validation, and Test Sets

Use three disjoint subsets:

- **Training set:** fit preprocessing parameters and model parameters.
- **Validation set:** compare models and select a probability threshold.
- **Test set:** estimate final performance once, after all modeling choices are locked.

Use a 60% / 20% / 20% split. Because the positive class is uncommon, use stratification.

```
set.seed(42)

# TODO: create train_data, validation_data, and test_data.
# Use initial_split() twice and stratify by damp_grey_soil.

dev_test_split <- initial_split(landsat,prop = 0.80,strata = damp_grey_soil) # 80% contains train and v
dev_data <- training(dev_test_split)
test_data <- testing(dev_test_split)
dim(dev_data)

## [1] 5148 37

train_val_split <- initial_split(dev_data,prop = 0.75,strata = damp_grey_soil) # now its total 60%/20%

validation_data=testing(train_val_split)
train_data=training(train_val_split)

dim(validation_data)==dim(test_data)# yay they are the same size

## [1] TRUE TRUE

(dim(test_data)[1]/nrow(landsat))==0.2

## [1] TRUE

# Hints:
# dev_test_split <- initial_split(...)
# dev_data <- training(...)
# test_data <- testing(...)
# train_val_split <- initial_split(...)

# A split summary showing rows, positive count, and positive fraction
# for train, validation, and test.

split_summary <- bind_rows(
  train_data %>%
```

```

summarize(
  split = "train",
  rows = n(),
  positive_count = sum(damp_grey_soil == "damp_grey_soil"),
  positive_fraction = mean(damp_grey_soil == "damp_grey_soil")
),
validation_data %>%
  summarize(
    split = "validation",
    rows = n(),
    positive_count = sum(damp_grey_soil == "damp_grey_soil"),
    positive_fraction = mean(damp_grey_soil == "damp_grey_soil")
  ),
test_data %>%
  summarize(
    split = "test",
    rows = n(),
    positive_count = sum(damp_grey_soil == "damp_grey_soil"),
    positive_fraction = mean(damp_grey_soil == "damp_grey_soil")
  )
)

split_summary

```

```

## # A tibble: 3 x 4
##   split      rows positive_count positive_fraction
##   <chr>    <int>         <int>         <dbl>
## 1 train     3861             360           0.0932
## 2 validation 1287             146           0.113
## 3 test      1287             120           0.0932

```

Question 4: Splitting and Leakage

Answer each prompt in 2-4 sentences.

1. Why is stratification useful in this dataset?
2. What is the validation set used for?
3. Why should the test set not be used to choose a model or threshold?
4. Explain how fitting scaling, imputation, or upsampling before splitting can leak information.

Answers:

1. Stratification is needed for this dataset because there are so few damp grey soil that random selection can easily put all samples into the testing dataset and mess up the model's fitting. Stratification makes every split of the dataset have an equivalent percentage of target variable, so that the model fitting isn't overcoming a poor data split.
2. Validation set is the 'baby' test dataset. After model's are fitted and cross validated, different model structures can be compared with the validation dataset so there is no leakage of data when the final testing is done.
3. Any steps that involve choosing the model or threshold should be done using the validation dataset. If the test dataset is used, we are giving the model the results of what we plan to test it on and this is a form of data leakage.
4. All these steps involve using multiple datapoints from the whole dataset to fill gaps, normalize, or increase resolution of data. Often this uses the mean or some kind of statistics to 'make up data'. Since

this uses data that may not be together in the test/train splits, the machine ‘gets an glance’ at the full dataset and has an easier time estimating the predictors since it has access to that data.

5. Build Logistic-Regression Workflows

We will compare three workflows:

1. A majority-class baseline model.
2. Ordinary logistic regression.
3. Logistic regression with upsampling applied inside the recipe.

Upsampling increases the number of minority-class examples in the training data by resampling them. It must occur inside the modeling workflow so that it is applied only to training data.

```
# Create a basic recipe for logistic regression.
```

```
logistic_recipe <- recipe(damp_grey_soil ~ ., data = train_data) %>%  
  step_zv(all_predictors()) %>%  
  step_normalize(all_numeric_predictors())  
  
upsample_recipe <- logistic_recipe %>%  
  step_upsample(damp_grey_soil, over_ratio = 1)
```

```
# Create a null model specification and a logistic regression specification.
```

```
null_spec <- null_model() %>%  
  set_engine("parsnip") %>%  
  set_mode("classification")  
  
logistic_spec <- logistic_reg() %>%  
  set_engine("glm") %>%  
  set_mode("classification")
```

```
# TODO: create three workflows named:
```

```
# baseline_workflow  
# logistic_workflow  
# upsampled_logistic_workflow
```

```
baseline_workflow <- workflow() |>  
  add_recipe(logistic_recipe) |> # I assume this is the basic recipe  
  add_model(null_spec)
```

```
logistic_workflow <- workflow() |>  
  add_recipe(logistic_recipe) |>  
  add_model(logistic_spec)
```

```
upsampled_logistic_workflow <- workflow() |>  
  add_recipe(upsample_recipe) |>  
  add_model(logistic_spec)
```

```
# Hint: use workflow(), add_recipe(), and add_model().
```

```
# TODO: fit all three workflows on train_data.
```

```
baseline_fit=fit(baseline_workflow, data=train_data)  
logistic_fit=fit(logistic_workflow, data=train_data)  
upsampled_logistic_fit=fit(upsampled_logistic_workflow, data=train_data)
```

baseline_fit

```
## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: null_model()
##
## -- Preprocessor -----
## 2 Recipe Steps
##
## * step_zv()
## * step_normalize()
##
## -- Model -----
## Null Regression Model
## Predicted Value: other
```

logistic_fit

```
## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: logistic_reg()
##
## -- Preprocessor -----
## 2 Recipe Steps
##
## * step_zv()
## * step_normalize()
##
## -- Model -----
##
## Call: stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
##
## Coefficients:
## (Intercept)      band_1      band_2      band_3      band_4      band_5
##      3.65874      0.06737      0.25113     -0.19706     -0.84498     -0.40310
##      band_6      band_7      band_8      band_9      band_10     band_11
##     -0.58620      0.11832      0.83001      0.48916      1.20737      0.18126
##      band_12     band_13     band_14     band_15     band_16     band_17
##     -0.75965      0.35786      0.24105     -0.12368      0.79427     -0.68738
##      band_18     band_19     band_20     band_21     band_22     band_23
##     -1.87006     -0.02271      1.22269     -0.60752     -1.04408      0.02176
##      band_24     band_25     band_26     band_27     band_28     band_29
##      1.21784      0.21549     -0.77679      0.59369     -0.10824     -0.08804
##      band_30     band_31     band_32     band_33     band_34     band_35
##      0.55186     -0.46242     -0.03410     -0.48194      1.34283     -0.31430
##      band_36
##     -0.15495
##
## Degrees of Freedom: 3860 Total (i.e. Null); 3824 Residual
## Null Deviance:      2394
## Residual Deviance: 1972 AIC: 2046
```

upsampled_logistic_fit *#such a big AIC*

```
## == Workflow [trained] =====
```

```

## Preprocessor: Recipe
## Model: logistic_reg()
##
## -- Preprocessor -----
## 3 Recipe Steps
##
## * step_zv()
## * step_normalize()
## * step_upsample()
##
## -- Model -----
##
## Call: stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
##
## Coefficients:
## (Intercept)      band_1      band_2      band_3      band_4      band_5
##    1.56847    0.08868    0.21434   -0.23367   -0.91461   -0.37632
##    band_6    band_7    band_8    band_9    band_10    band_11
##   -0.55382    0.14940    0.81332    0.13281    1.32258    0.25999
##    band_12    band_13    band_14    band_15    band_16    band_17
##   -0.46487    0.25249    0.67215   -0.16954    0.54221   -0.85047
##    band_18    band_19    band_20    band_21    band_22    band_23
##   -2.11959    0.05334    1.35098   -0.60267   -0.72857    0.11363
##    band_24    band_25    band_26    band_27    band_28    band_29
##    0.94633    0.37605   -1.23681    0.90812   -0.33342    0.09242
##    band_30    band_31    band_32    band_33    band_34    band_35
##    0.78539   -0.99142    0.29233   -0.65915    1.14949   -0.11829
##    band_36
##   -0.05629
##
## Degrees of Freedom: 7001 Total (i.e. Null);  6965 Residual
## Null Deviance:      9707
## Residual Deviance: 7178  AIC: 7252
# Hints:
# baseline_fit <- fit(...)
# etc
# etc

```

Question 5: Model Choice

Answer in 1-3 sentences.

- Why include a majority-class baseline?
- Why is logistic regression a reasonable first classification model?
- What does upsampling change?
- Why must upsampling happen inside the training workflow?

Answer:

TODO:

A majority-class baseline model will give us a ‘control’ model that will show how changing the model and the sampling recipe will impact the model and how they fit better or worse than the baseline model. A logistic regression can calculate the probability between two categorical variables, which is ideal for classification. Upsampling increases the resolution of a dataset by interpolating between previously recorded points. Upsampling has to happen in the training workflow to prevent data leakage into the test datasets, and

the model should be able to run on data as it was recorded with minimal processing so it can truly predict.

6. Evaluate Models on the Validation Set

For an imbalanced classification problem, no single metric is sufficient.

Important metrics:

- **Recall / sensitivity:** fraction of damp-grey-soil pixels detected.
- **Precision:** fraction of predicted damp-grey-soil pixels that were actually damp grey soil.
- **Specificity:** fraction of other land-cover pixels correctly identified.
- **Balanced accuracy:** mean of sensitivity and specificity.
- **F1:** balances precision and recall equally.
- **F2:** gives recall more weight than precision.
- **ROC-AUC:** ranking performance across thresholds.
- **PR-AUC:** precision-recall area; often useful for imbalanced data.

```
positive_level <- "damp_grey_soil"
prob_col <- ".pred_damp_grey_soil"

predict_with_threshold <- function(fit, new_data, threshold = 0.5) {
  # TODO: return a data frame with truth, predicted probabilities, and thresholded classes.
  dev_predict=predict(fit, new_data=new_data, type='prob')>%
  bind_cols(new_data %>% select(damp_grey_soil))
  dev_predict=dev_predict%>%
  mutate(.pred_class = if_else(.pred_damp_grey_soil >= threshold, "damp_grey_soil", "other"))>%
  mutate(.pred_class = factor(.pred_class, levels = levels(damp_grey_soil)))

  return(dev_predict)

  # Hints:
  # predict(...) #Only need to update this line!!
  # bind_cols(new_data %>% select(damp_grey_soil))
  # mutate(.pred_class = if_else(.pred_damp_grey_soil >= threshold, "damp_grey_soil", "other"))
  # mutate(.pred_class = factor(.pred_class, levels = levels(damp_grey_soil)))
}

calculate_metrics <- function(predictions) {
  tibble(
    accuracy = accuracy_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class
    ),
    recall = sens_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
      event_level = "first"
    ),
    precision = precision_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
      event_level = "first"
    ),
    specificity = spec_vec(
```

```

    predictions$damp_grey_soil,
    predictions$.pred_class,
    event_level = "first"
  ),
  balanced_accuracy = bal_accuracy_vec(
    predictions$damp_grey_soil,
    predictions$.pred_class,
    event_level = "first"
  ),
  f1 = f_meas_vec(
    predictions$damp_grey_soil,
    predictions$.pred_class,
    event_level = "first"
  ),
  f2 = f_meas_vec(
    predictions$damp_grey_soil,
    predictions$.pred_class,
    beta = 2,
    event_level = "first"
  ),
  roc_auc = roc_auc_vec(
    predictions$damp_grey_soil,
    predictions$.pred_damp_grey_soil,
    event_level = "first"
  ),
  pr_auc = pr_auc_vec(
    predictions$damp_grey_soil,
    predictions$.pred_damp_grey_soil,
    event_level = "first"
  )
)
}

```

```

# TODO: calculate validation metrics for all three models at threshold 0.5.

```

```

baseline_val <- predict_with_threshold(baseline_fit, new_data=validation_data,threshold = 0.5)
logistic_val <- predict_with_threshold(logistic_fit, new_data=validation_data,threshold = 0.5)
upsampled_val <- predict_with_threshold(upsampled_logistic_fit, new_data=validation_data,threshold = 0.5)

```

```

# Hints:
# baseline_val <- predict_with_threshold(...)
# logistic_val <- predict_with_threshold(...)
# upsampled_val <- predict_with_threshold(...)
#
bind_rows(
  calculate_metrics(baseline_val) %>% mutate(model = "Baseline"),
  calculate_metrics(logistic_val) %>% mutate(model = "Logistic regression"),
  calculate_metrics(upsampled_val) %>% mutate(model = "Upsampled logistic regression")
)

```

```

## Warning: While computing binary `precision()`, no predicted events were detected (i.e.

```

```

## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil
## While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil
## While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil

## # A tibble: 3 x 10
##   accuracy recall precision specificity balanced_accuracy    f1    f2
##   <dbl> <dbl>    <dbl>    <dbl>          <dbl> <dbl> <dbl>
## 1   0.887 0        NA        1            0.5    NA    NA
## 2   0.886 0.0274   0.444    0.996        0.512 0.0516 0.0337
## 3   0.642 0.801    0.213    0.621        0.711 0.337  0.516
## # i 3 more variables: roc_auc <dbl>, pr_auc <dbl>, model <chr>

# TODO: create confusion matrices for the two logistic-regression models.
logistic_conf=conf_mat(data = logistic_val, truth = damp_grey_soil, estimate = .pred_class)
upsample_conf=conf_mat(data = upsampled_val, truth = damp_grey_soil, estimate = .pred_class)
logistic_conf

##           Truth
## Prediction  damp_grey_soil other
## damp_grey_soil           4     5
## other           142    1136

upsample_conf

##           Truth
## Prediction  damp_grey_soil other
## damp_grey_soil           117   432
## other           29     709

# Hint:
# conf_mat(...)

```

Question 6: Validation Comparison

Using the validation results, answer in complete sentences.

1. Why can the baseline model have high accuracy but zero usefulness for the positive class?
2. Compare ordinary logistic regression and upsampled logistic regression at threshold 0.5.
3. Which model has better recall for damp grey soil?
4. What happened to precision and specificity?

Answers:

1. The baseline model has a high accuracy for predicting the class 'other'. This model did not once correctly predict 'damp grey soil' and that is why it has zero usefulness.
2. While the logistic regression has a higher accuracy, the upsampled logistic regression has much better recall, which is preferred when modeling for rare cases.

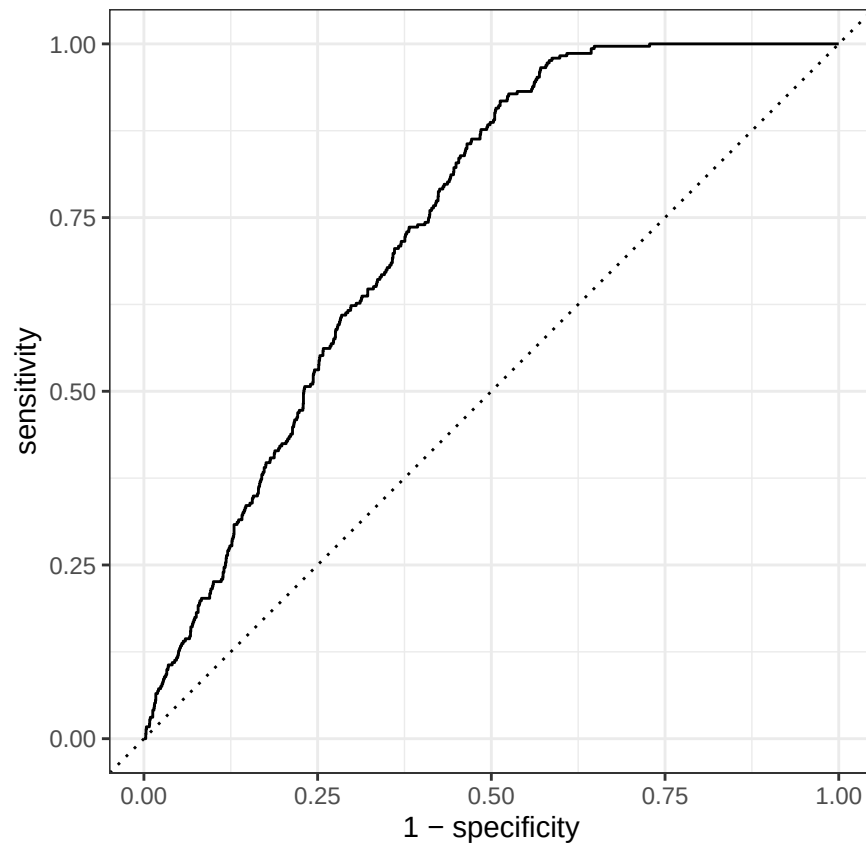
3. The upsampled logistic regression has the highest recall (by a lot).
4. Precision and specificity steeply drops between the original logistic and the upsampled logistic regressions. Specificity is highest in the baseline model, but it has no precision.

7. Examine ROC and Precision-Recall Curves

Curves show model behavior across possible thresholds. For imbalanced classification, the precision-recall curve is often more informative than the ROC curve because it focuses directly on positive-class detection.

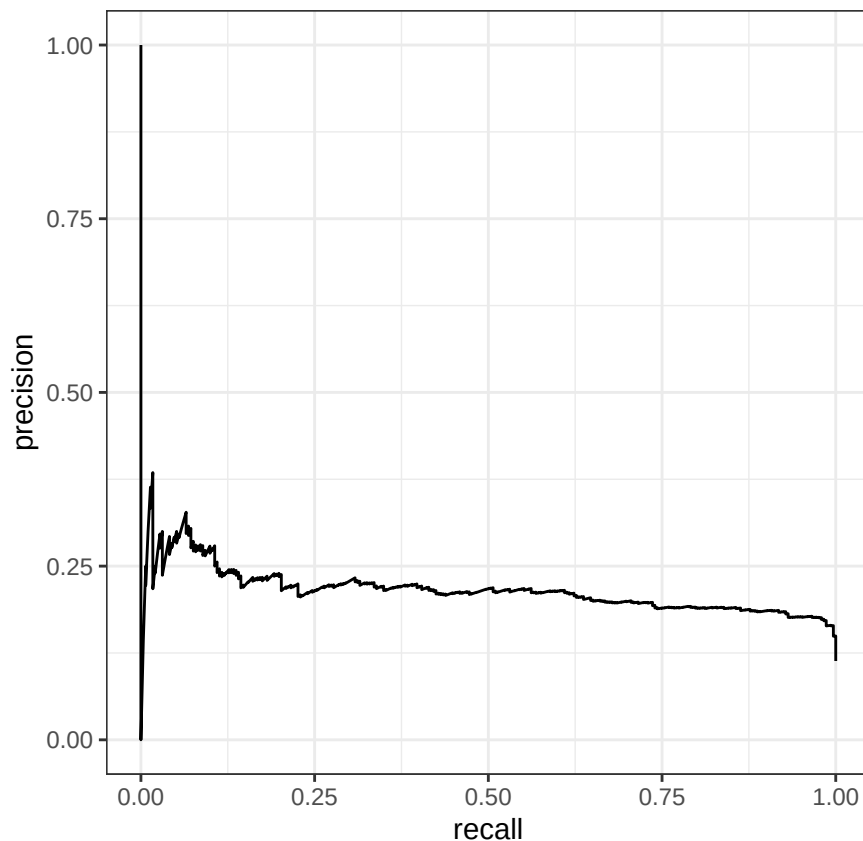
plot ROC curves for ordinary and upsampled logistic regression.

```
bind_rows(
  logistic_val %>% mutate(model = "Logistic regression"),
  upsampled_val %>% mutate(model = "Upsampled logistic regression")
) %>%
  roc_curve(truth = damp_grey_soil, .pred_damp_grey_soil, event_level = "first") %>%
  autoplot()
```



TODO: plot precision-recall curves for ordinary and upsampled logistic regression.

```
bind_rows(
  logistic_val %>% mutate(model = "Logistic regression"),
  upsampled_val %>% mutate(model = "Upsampled logistic regression")
) %>%
  pr_curve(truth = damp_grey_soil, .pred_damp_grey_soil, event_level = "first") %>%
  autoplot()
```



Hint: use `pr_curve()` and `autoplot()`.

Question 7: Curve Interpretation

Answer in 3-6 sentences.

- Why is the precision-recall baseline equal to the positive-class prevalence?
- Can two models have similar ROC-AUC but different rare-class precision? Why?
- Which curve is more useful for thinking about missed rare land-cover areas versus false alarms?

Answer:

TODO:

If the model is predicting correctly, as proportional to the dataset size, the precision-recall baseline will equal positive-class prevalence. The precision-recall baseline also better counts for thresholds. ROU-AUC are showing how good the model is at randomly guessing and being correct by focusing on true and false positives. Rare-class precision is going to focus on having a high recall to prevent false negatives. Two different models can have similar percentages of true/false positives, but like the upsampled logistic regression model, is focused on having higher rare-class precision. I think the precision-recall baseline is useful for missing rare land-cover areas as those are false negatives. False alarms would be addressed with ROU-AUC.

8. Test Probability Thresholds Using Validation Data

The default threshold of 0.5 is not automatically best for an imbalanced problem, but changing the threshold does not improve the model itself. It only changes the tradeoff between the types of errors the model makes.

For this exercise, test a range of thresholds for the **upsampled logistic regression** model and identify the threshold that maximizes **F2** on the validation set. Then compare that threshold with 0.5. If the F2-selected

threshold creates too many false positives, lowers balanced accuracy, or makes the model less useful for the stated mapping goal, it is reasonable to conclude that threshold adjustment did not help.

The important lesson is not that threshold tuning always improves performance. The lesson is that thresholds encode a decision policy: they decide whether the model should miss more positive cases or produce more false alarms.

```
# Search thresholds from 0.01 to 0.99 and identify the threshold with the best F2.
```

```
threshold_grid <- tibble(threshold = seq(0.01, 0.99, by = 0.01))
threshold_results <- threshold_grid %>%
  mutate(predictions = map(threshold, ~ predict_with_threshold(upsampled_logistic_fit, validation_data,
    ~ predict_with_threshold(.x, validation_data, .y)))) %>%
  mutate(metrics = map(predictions, calculate_metrics)) %>%
  select(-predictions) %>%
  unnest(metrics)
```

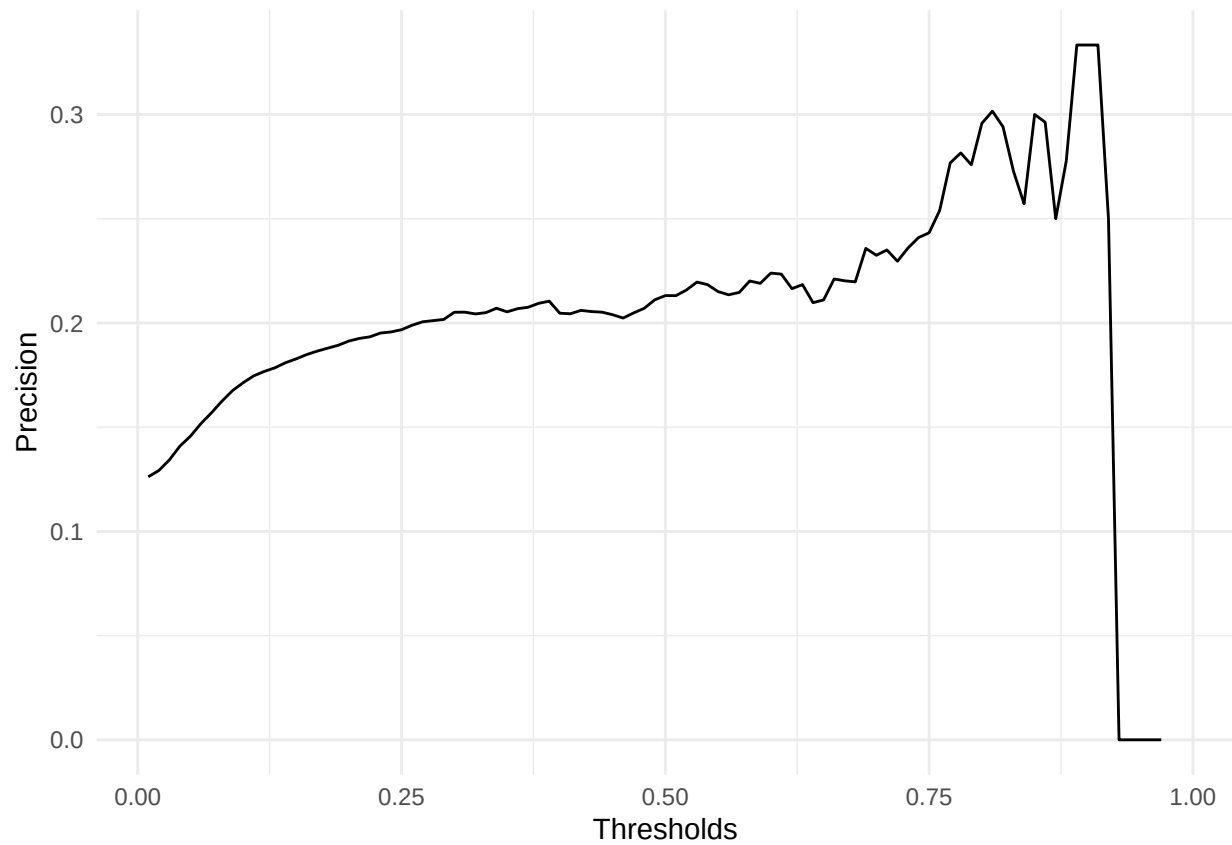
```
## Warning: There were 6 warnings in `mutate()`.
## The first warning was:
## i In argument: `metrics = map(predictions, calculate_metrics)`.
## Caused by warning:
## ! While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil
## i Run `dplyr::last_dplyr_warnings()` to see the 5 remaining warnings.
```

```
best_threshold <- threshold_results %>%
  slice_max(f2, n = 1, with_ties = FALSE) %>%
  pull(threshold)
```

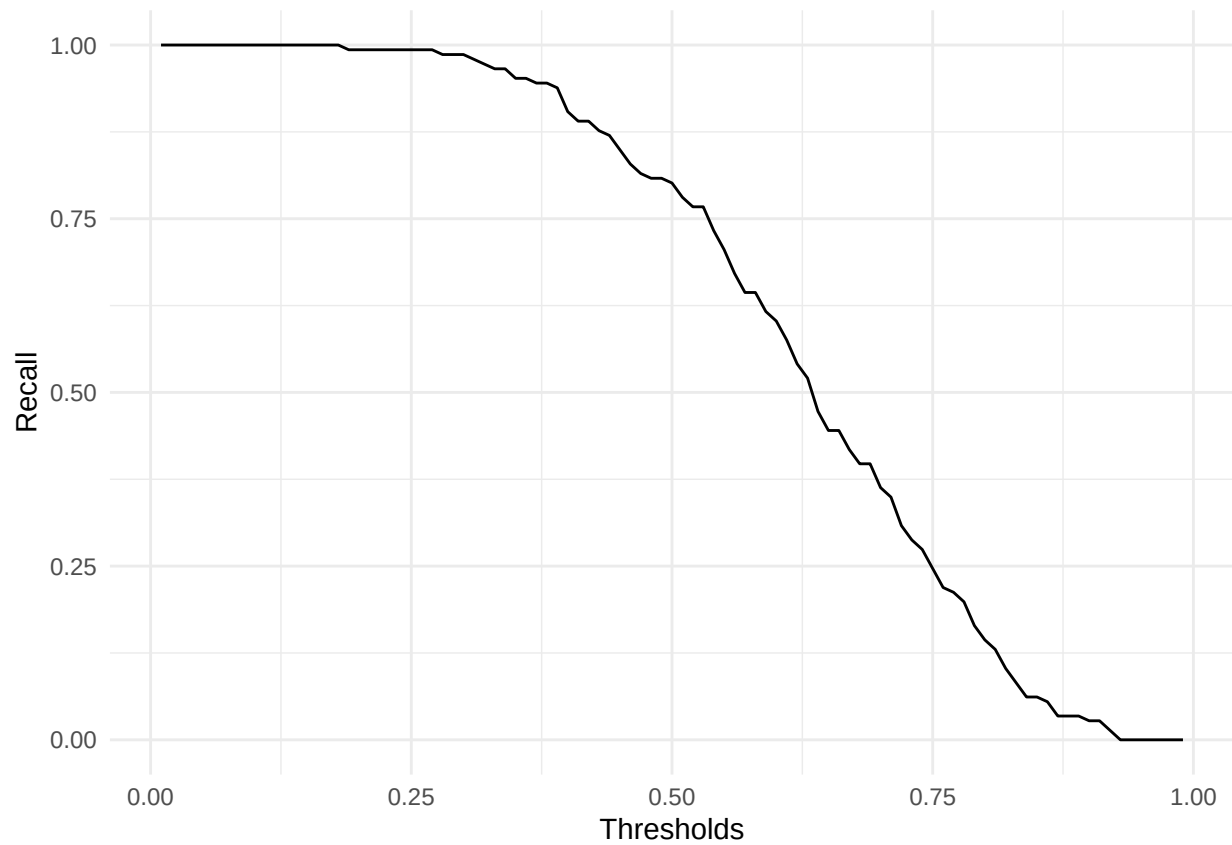
```
# TODO: plot precision, recall, and F2 across thresholds.
```

```
# I think these variables came in the long format ready to plot.
ggplot()+
  geom_line(data=threshold_results, aes(x=threshold, y=precision))+
  labs(x="Thresholds", y="Precision")
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

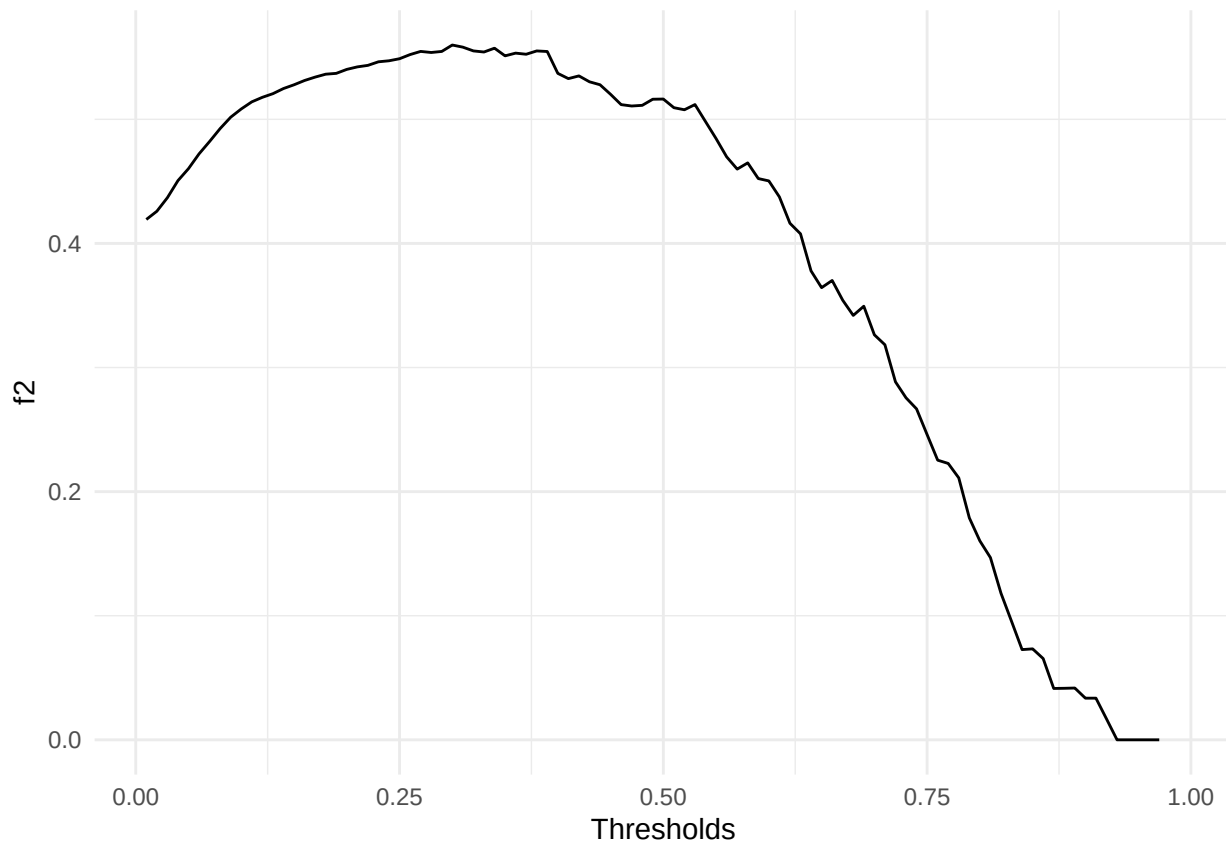


```
ggplot()+  
  geom_line(data=threshold_results,aes(x=threshold, y=recall))+  
  labs(x="Thresholds",y="Recall")
```



```
ggplot()+  
  geom_line(data=threshold_results,aes(x=threshold, y=f2))+  
  labs(x="Thresholds",y="f2")
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range  
## (`geom_line()`).
```



Hint: pivot precision, recall, and f2 into long format, then use ggplot().

```
logistic_val2 <- predict_with_threshold(logistic_fit, new_data=validation_data, threshold = 0.38)
upsampled_val2 <- predict_with_threshold(upsampled_logistic_fit, new_data=validation_data, threshold = 0.38)
logistic_conf2=conf_mat(data = logistic_val2, truth = damp_grey_soil, estimate = .pred_class)
upsample_conf2=conf_mat(data = upsampled_val2, truth = damp_grey_soil, estimate = .pred_class)
logistic_conf2
```

```
##           Truth
## Prediction   damp_grey_soil other
## damp_grey_soil           10    13
## other              136   1128
```

```
upsample_conf2
```

```
##           Truth
## Prediction   damp_grey_soil other
## damp_grey_soil           138   521
## other              8    620
```

Question 8: Threshold Choice

Answer in complete sentences.

1. Is your selected threshold above or below 0.5?
2. What happens to false positives and false negatives when the threshold is lowered?

3. Did the F2-selected threshold make the model more useful than the 0.5 threshold? Explain using at least two metrics or confusion-matrix counts.
4. Is maximizing F2 automatically the correct policy? What field, cost, or scientific information would you need before choosing a real operational threshold?

Answers:

1. My threshold is below 0.5. Best threshold ended up as 0.38.
2. False negatives are decreased when the threshold is lowered (recall is high). There are also more false positives when the threshold is lowered from 0.5 (precision is lower, meaning the denominator/FP are higher).
3. I would argue that in the rare-class focused model, the F2 selected threshold did make the model more useful. Looking at the confusion-matrix counts, the models (both logistic and upsampled logistic) saw an increase in True positives and False positives and a decrease in False negatives and True negatives. The upsampled logistic regression model specifically saw little change in true-damp_grey_soil and predicted-damp_grey_soil, and true-other and predicted-other. This model had a relatively huge decrease from the previous threshold in false negatives (true-damp_grey_soil and predicted-other), 30 to 8. There was a large increase in false positives (true-other and predicted-damp_grey_soil), 463 to 541. For the rare-class example we are looking at, I think decreasing the missed detections (false negatives) is more important than having high false positives.
4. Maximizing F2 is not automatically the correct policy. As max F2 is trying to increase false positives, which is good for cases where to test again is affordable and we are looking for minority cases, like in the medical field or identifying risk.

9. Final Evaluation on the Untouched Test Set

At this point, the model and threshold have been selected using validation data. If the F2-selected threshold made validation performance worse for the assignment goal, you may keep the default 0.5 threshold instead. State which threshold you chose and why.

Do not use the test result to go back and choose a different model or threshold.

```
# Evaluate the upsampled logistic regression model on the test set
# using the threshold chosen from validation data.
```

```
final_test_predictions <- predict_with_threshold(upsampled_logistic_fit, test_data, threshold = best_th)
calculate_metrics(final_test_predictions)
```

```
## # A tibble: 1 x 9
##   accuracy recall precision specificity balanced_accuracy    f1    f2 roc_auc
##   <dbl> <dbl>    <dbl>    <dbl>          <dbl> <dbl> <dbl>  <dbl>
## 1    0.531  0.983    0.164    0.484          0.734 0.281 0.492  0.770
## # i 1 more variable: pr_auc <dbl>
```

```
conf_mat(final_test_predictions, truth = damp_grey_soil, estimate = .pred_class)
```

```
##               Truth
## Prediction      damp_grey_soil other
## damp_grey_soil          118    602
## other                   2    565
```

Question 9: Methods and Limitations

Answer in a short paragraph.

1. Compare upsampling with class weighting and random undersampling.

2. Where must resampling occur to avoid leakage when cross-validation is used?

Answer:

TODO:

Upsampling is beneficial as it increases the presences of the minority class data without reducing the sample size (random undersampling) and changes the total fit of the model instead of adjusting during validation (class weighting). This method allows us to use the whole dataset in the splits and can be applied to the cross validation. In this example (and I assume is the correct way) the resampling occurs just after the dataset is split into test/train/validation, and it is applied while the model it being fit to the training set. This fit is applied to the validation and test when predicting off these new datasets.

Grading Rubric (25 points)

Component	Points
Reproducibility, loading, and data checks	2
Class-imbalance analysis	3
Correct train/validation/test split and leakage explanation	3
Logistic-regression workflows	3
Baseline and model fitting	2
Validation metrics and interpretation	3
ROC and precision-recall curve interpretation	2
Threshold comparison using validation data	2
Final test evaluation	2
Scientific interpretation and limitations	3

Reproducibility Checklist

Before submitting, confirm that:

- ☐ The document knits without manual state from earlier sessions.
- ☐ All **TODO** sections are complete.
- ☐ Random seeds are fixed.
- ☐ Preprocessing is fitted only through workflows on training data.
- ☐ Upsampling occurs only inside the recipe/workflow.
- ☐ Validation data, not test data, are used for model and threshold decisions.
- ☐ The test set is evaluated only in the final section.
- ☐ Metrics are interpreted for the positive class, damp grey soil.
- ☐ Figures have readable titles and axis labels.
- ☐ Scientific limitations and error costs are discussed.

Dataset Citation

Srinivasan, A. (1993). Statlog (Landsat Satellite) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C55887>